Semana 9: Python científico — NumPy, Pandas y Matplotlib

Nivel: Intermedio | **Prerrequisitos:** Semana 8 (media, mediana, desviación estándar, normalización), Semana 7 (vectores, producto punto, matrices), Semana 6 (pruebas con `pytest`, `pytest.approx`), Semana 4 (listas y diccionarios) y Semana 3 (funciones) | **Versión:** v1.0 | **Fecha:** 2026-10-19

Objetivos de aprendizaje

1. Crear y manipular **arreglos de NumPy** (`ndarray`): forma, tipo, indexación y filtrado con máscaras booleanas.
2. Explicar qué es la **vectorización** y por qué reemplaza a los bucles `for` en código numérico.
3. Cargar, limpiar y resumir una **tabla con Pandas** (`DataFrame`): columnas, filtros, valores faltantes y `groupby`.
4. Producir un **gráfico claro con Matplotlib** (título, etiquetas de ejes) y guardarlo en un archivo.
5. Reconocer cómo estas tres librerías forman la base de todo el ecosistema de ML.

---

1. Por qué existen estas librerías

Hasta ahora todo se programó en Python puro: listas, bucles, módulos estándar. Funciona, pero tiene dos límites cuando los datos crecen:

- **Velocidad:** una lista de Python guarda objetos sueltos y un bucle `for` los recorre de a uno. Con un millón de números, es lento.
- **Comodidad:** en la Semana 7 y la 8 escribiste a mano el producto punto, la media y la normalización. Con datos reales eso se vuelve repetitivo.

Analogía: la cocina de casa y la cocina industrial. Con Python puro cocinas plato por plato. NumPy es una línea de producción: la misma operación se aplica a *toda la bandeja a la vez*. Pandas es el cuaderno de pedidos con columnas con nombre. Matplotlib es la vitrina donde muestras el resultado.

Librería	Qué te da	Se parece a…
NumPy	Arreglos numéricos rápidos de N dimensiones	Vectores y matrices de la Semana 7

Librería	Qué te da	Se parece a...
Pandas	Tablas con columnas con nombre (`DataFrame`)	Un diccionario de listas (Semana 4) con superpoderes
Matplotlib	Gráficos	Convertir números en una imagen

Estas librerías **no vienen con Python**; se instalan una vez (`pip install numpy pandas matplotlib`) y de preferencia dentro de un entorno virtual por proyecto.

---

2. NumPy: arreglos y vectorización

Del vector a `ndarray`

En la Semana 7 un vector era una lista de números. En NumPy es un **arreglo** (`ndarray`): todos sus elementos tienen el mismo tipo, así que se guarda compacto y se opera rápido.

```
import numpy as np

sueldos = np.array([1, 2, 3, 4, 1000])    # a partir de una lista (Semana 4)

print(sueldos.shape)    # (5,)  → forma: 5 elementos en una dimensión
print(sueldos.dtype)    # int64 → todos los elementos comparten tipo
print(sueldos.mean())   # 202.0 → la media de la Semana 8, sin escribir la función
print(np.median(sueldos))   # 3.0
print(sueldos * 2)      # [   2    4    6    8 2000] → ¡sin bucle!
```

Fíjate en la última línea: `sueldos * 2` multiplica **cada elemento**. Con listas, `[1, 2, 3] * 2` repite la lista; con NumPy opera número por número. Eso es la **vectorización**: escribir la operación sobre el arreglo completo y dejar que NumPy haga el bucle, por dentro, en código compilado muy rápido.

Matrices, ejes y producto punto

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])       # lista de listas (Semana 7) → matriz 2×3

print(m.shape)          # (2, 3) → 2 filas, 3 columnas
print(m.sum(axis=0))    # [5 7 9]  → suma bajando por las columnas
print(m.sum(axis=1))    # [ 6 15]  → suma a lo largo de cada fila

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a @ b)            # 32 → producto punto de la Semana 7, con el operador @
```

> **Truco para el `axis`:** `axis=0` "colapsa" las filas (el resultado tiene una entrada por columna); `axis=1` colapsa las columnas (una entrada por fila).

Filtrar con máscaras booleanas

```
print(sueldos > 3)             # [False False False  True  True]
print(sueldos[sueldos > 3])    # [   4 1000] → solo los que cumplen la condición
```

Es el equivalente de una *list comprehension* con `if` (Semana 4), pero sin bucle explícito.

Vectorizar es más rápido (y más corto)

```
rng = np.random.default_rng(42)       # generador con semilla (Semana 8)
x = rng.random(1_000_000)             # un millón de números en [0, 1)

# Suma de cuadrados con un bucle de Python:
lento = sum(v * v for v in x.tolist())
# La misma cuenta, vectorizada (producto punto del arreglo consigo mismo):
rapido = float(np.dot(x, x))
```

Ambos dan `333336.885…`; la versión vectorizada suele ser decenas de veces más rápida (en la verificación de esta clase lo fue por más de un factor 10). Regla práctica: **si estás escribiendo un `for` sobre un arreglo de NumPy, probablemente exista una forma vectorizada.**

Broadcasting: operar arreglos de distinta forma

```
notas = np.array([[7, 8],
                  [5, 6],
                  [9, 10]])               # 3 alumnos × 2 exámenes
print(notas - notas.mean(axis=0))
# [[ 0.  0.]
#  [-2. -2.]
#  [ 2.  2.]]   → a cada nota se le resta el promedio de SU examen
```

NumPy "estira" automáticamente la media (forma `(2,)`) para restarla a cada fila. Con esto, la **normalización** de la Semana 8 es una sola línea: `(v - v.mean()) / v.std()`.

---

3. Pandas: tablas con nombre

Un `DataFrame` es una tabla: filas numeradas, columnas con nombre y un tipo por columna. Por dentro, cada columna es un arreglo de NumPy; por eso todo lo anterior se hereda.

```
import pandas as pd

df = pd.DataFrame({                     # un diccionario de listas (Semana 4)
    "producto": ["pan", "leche", "pan", "queso", "leche", "pan", "queso", "leche"],
    "tienda":   ["A", "A", "B", "B", "A", "A", "B", "B"],
    "unidades": [10, 4, 7, 2, 5, 12, 3, None],     # None → dato faltante
    "precio":   [1.5, 2.0, 1.5, 8.0, 2.0, 1.5, 8.0, 2.0],
})
```

Explorar y limpiar

```
print(df.isna().sum())      # cuántos faltantes por columna → unidades: 1
df["unidades"] = df["unidades"].fillna(df["unidades"].median())   # rellenar con la mediana
```

Los datos reales **siempre** traen huecos. Rellenar con la mediana (y no la media) es la decisión de la Semana 8: la mediana no se deja arrastrar por valores extremos.

Columnas, filtros y resúmenes

```
df["total"] = df["unidades"] * df["precio"]     # nueva columna, vectorizada

print(df[df["total"] > 15])      # filas cuyo total supera 15 (máscara booleana)
print(df.describe())             # conteo, media, desviación, mínimo, cuartiles, máximo
```

`groupby`: agrupar y resumir

```
por_producto = df.groupby("producto")["total"].sum().sort_values(ascending=False)
print(por_producto)
# producto
# pan      43.5
# queso    40.0
# leche    28.0
```

`groupby` sigue el patrón **separar → aplicar → combinar**: separa las filas por producto, suma cada grupo y junta los resultados. Es el equivalente de construir a mano un diccionario de acumuladores (Semana 4), pero en una línea.

---

4. Matplotlib: ver los datos

Un número resume; un gráfico *revela*. Un gráfico mal rotulado, en cambio, confunde: siempre pon **título y etiquetas de ejes**.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4))      # figura (el lienzo) y ejes (el gráfico)
ax.bar(por_producto.index, por_producto.values, color="#3548C4")
ax.set_title("Ventas por producto")
ax.set_xlabel("Producto")
ax.set_ylabel("Total vendido ($)")
fig.tight_layout()
fig.savefig("ventas.png", dpi=150)
plt.close(fig)                              # libera la memoria de la figura
```

Se usa la interfaz orientada a objetos (`fig, ax = plt.subplots()`), recomendada por la propia documentación, en lugar de llamar funciones sueltas de `plt.` que dependen de un "gráfico

actual" implícito. Para scripts y servidores sin pantalla, se fija el *backend* con `matplotlib.use("Agg")` antes de importar `pyplot`.

---

5. Por qué esto importa para IA/ML

- **Todo el ecosistema se apoya en NumPy.** scikit-learn recibe y devuelve arreglos de NumPy; PyTorch y TensorFlow usan *tensores* con la misma idea (arreglos N-dimensionales con operaciones vectorizadas) y convierten fácilmente desde/hacia NumPy.
- **Un conjunto de datos de ML es una matriz.** Cada fila es un ejemplo y cada columna una característica (*feature*): una matriz `n_ejemplos × n_características`, exactamente las matrices de la Semana 7.
- **El trabajo real es preparar datos.** Limpiar faltantes, filtrar, agrupar y normalizar con Pandas/NumPy suele ser la mayor parte del tiempo de un proyecto, antes de entrenar nada.
- **Graficar es depurar.** Ver la distribución de los datos o la curva de error de entrenamiento (Semana 7) en un gráfico detecta problemas que los números no muestran.
- **Vectorizar es lo que hace posible entrenar.** Los modelos multiplican matrices enormes; sin operaciones vectorizadas (y luego GPU) el entrenamiento sería inviable. Es la razón de rendimiento detrás de lo visto en la Semana 5.

---

6. Ejemplo práctico: análisis de ventas de una tienda

Problema: dada una tabla de ventas con un dato faltante, limpiarla, calcular cuánto se vendió por producto y graficarlo.

Pseudocódigo

```
cargar la tabla de ventas
rellenar las unidades faltantes con la mediana de las unidades
total de cada venta = unidades × precio
sumar los totales agrupando por producto
ordenar de mayor a menor
dibujar un gráfico de barras con título y etiquetas
```

Código real (Python)

```
# analisis_ventas.py
import matplotlib
matplotlib.use("Agg")  # backend sin ventana: sirve en servidores y scripts
import matplotlib.pyplot as plt
import pandas as pd


def cargar_ventas() -> pd.DataFrame:
    """Datos de ejemplo; en la vida real: pd.read_csv("ventas.csv")."""
```

```
    return pd.DataFrame({
        "producto": ["pan", "leche", "pan", "queso", "leche", "pan", "queso", "leche"],
        "tienda": ["A", "A", "B", "B", "A", "A", "B", "B"],
        "unidades": [10, 4, 7, 2, 5, 12, 3, None],
        "precio": [1.5, 2.0, 1.5, 8.0, 2.0, 1.5, 8.0, 2.0],
    })


def limpiar(df: pd.DataFrame) -> pd.DataFrame:
    """Devuelve una copia con los faltantes de 'unidades' rellenados con la mediana."""
    limpio = df.copy()  # no mutar el original
    limpio["unidades"] = limpio["unidades"].fillna(limpio["unidades"].median())
    return limpio


def ventas_por_producto(df: pd.DataFrame) -> pd.Series:
    """Total vendido (unidades * precio) por producto, de mayor a menor."""
    total = df["unidades"] * df["precio"]
    return total.groupby(df["producto"]).sum().sort_values(ascending=False)


def graficar(totales: pd.Series, ruta: str) -> None:
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(totales.index, totales.values, color="#3548C4")
    ax.set_title("Ventas por producto")
    ax.set_xlabel("Producto")
    ax.set_ylabel("Total vendido ($)")
    fig.tight_layout()
    fig.savefig(ruta, dpi=150)
    plt.close(fig)  # liberar memoria


if __name__ == "__main__":
    totales = ventas_por_producto(limpiar(cargar_ventas()))
    print(totales)
    graficar(totales, "ventas.png")
```

Resultado verificado por ejecución: `pan 43.5`, `queso 40.0`, `leche 28.0`, y se crea `ventas.png`. Observa las decisiones de ingeniería: funciones pequeñas con una sola responsabilidad (Semana 3), anotaciones de tipo, y `limpiar` devuelve una **copia** para no alterar los datos originales (Semana 6).

Pruebas con pytest (Semana 6)

```
# test_analisis_ventas.py
import numpy as np
import pandas as pd
import pytest
from analisis_ventas import cargar_ventas, limpiar, ventas_por_producto


def test_limpiar_rellena_con_la_mediana():
    limpio = limpiar(cargar_ventas())
    assert limpio["unidades"].isna().sum() == 0
    assert limpio["unidades"].iloc[-1] == 5
```

```
def test_limpiar_no_modifica_el_original():
    original = cargar_ventas()
    limpiar(original)
    assert original["unidades"].isna().sum() == 1


def test_ventas_por_producto():
    totales = ventas_por_producto(limpiar(cargar_ventas()))
    assert totales.index[0] == "pan"
    assert totales["pan"] == pytest.approx(43.5)
    assert totales.sum() == pytest.approx(111.5)


def test_normalizacion_numpy():
    v = np.array([2, 4, 4, 4, 5, 5, 7, 9], dtype=float)
    z = (v - v.mean()) / v.std()
    assert z.mean() == pytest.approx(0.0)
    assert z.std() == pytest.approx(1.0)


def test_vectorizar_equivale_al_bucle():
    x = np.arange(1000, dtype=float)
    assert float(x @ x) == pytest.approx(sum(v * v for v in x.tolist()))
```

Las cinco pruebas pasan al ejecutar `pytest` (verificado antes de publicar; versiones usadas: NumPy 2.4, Pandas 3.0, Matplotlib 3.11). Para comparar decimales y arreglos se usa `pytest.approx` (Semana 7). En proyectos con muchas pruebas numéricas también existe `numpy.testing.assert_allclose`.

---

7. Notas de vigencia técnica (2026)

- **Pandas 3.0:** es la versión vigente y trae cambios que rompen código antiguo. Las columnas de texto usan por defecto un tipo de cadena dedicado (`str`) en lugar de `object`, y rige *copy-on-write*: un `DataFrame` derivado se comporta siempre como una copia independiente. Consecuencia práctica: el viejo patrón de *encadenar asignaciones* (`df[df["a"] > 1]["b"] = 0`) ya no funciona; se asigna con `df.loc[condición, "b"] = 0`. Y sobre la advertencia clásica `SettingWithCopyWarning`: ya no existe en este modelo.
- **Evita `inplace=True`.** Está desaconsejado y no aporta ahorro de memoria; el estilo actual es reasignar (`df["x"] = df["x"].fillna(...)`) o encadenar métodos.
- **NumPy 2.x:** la API de aleatoriedad vigente es `np.random.default_rng(semilla)`; las funciones globales antiguas (`np.random.seed`, `np.random.rand`) se consideran heredadas. Además, NumPy 2 limpió alias antiguos de la API, por lo que tutoriales muy viejos pueden fallar.
- **`std` no significa lo mismo en todas partes:** NumPy calcula por defecto la desviación **poblacional** (`ddof=0`, la de la Semana 8) y Pandas la **muestral** (`ddof=1`). Por eso `arreglo.std()` y `serie.std()` difieren ligeramente; si importa, indícalo con `ddof`.
- **Alternativas al alza:** *Polars* es una librería de DataFrames moderna, muy rápida y con ejecución perezosa; es una opción real para datos grandes, pero Pandas sigue siendo el estándar que encontrarás en casi todo el ecosistema de ML, por lo que es lo que se

enseña aquí. Para gráficos interactivos existen Plotly y Altair; Matplotlib sigue siendo la base (Seaborn se construye encima).
- **Entornos:** crea un entorno virtual por proyecto (`python -m venv .venv`) y fija versiones en un `requirements.txt` o `pyproject.toml`; así tu análisis es reproducible (Semana 6).

---

8. Errores comunes de principiante

- Escribir un `for` para sumar o multiplicar elementos de un arreglo de NumPy, en lugar de vectorizar.
- Confundir `lista * 2` (repite la lista) con `arreglo * 2` (multiplica cada elemento).
- Confundir `axis=0` con `axis=1` en sumas y medias.
- Olvidar que los datos reales traen valores faltantes y llamar a funciones sin revisarlos (`df.isna().sum()` primero).
- Modificar el `DataFrame` original dentro de una función sin avisar; devuelve una copia.
- Usar `inplace=True` o asignaciones encadenadas (`df[cond]["col"] = …`); usa `.loc`.
- Publicar un gráfico sin título, sin etiquetas de ejes o sin unidades.
- Olvidar `plt.close(fig)` al generar muchos gráficos en un bucle (consume memoria).
- Instalar librerías "globales" para todo en lugar de un entorno virtual por proyecto.

---

9. Ejercicio propuesto

Con el `DataFrame` de ventas de esta clase, escribe `ventas_por_tienda(df)` que devuelva el total vendido por tienda y el **producto más vendido de cada tienda** (pista: `groupby` con dos columnas, `["tienda", "producto"]`, y luego `idxmax()` por tienda). Luego haz un gráfico de barras agrupadas (una barra por producto, una serie por tienda) con título, ejes rotulados y leyenda. Añade pruebas con `pytest` y versiona el resultado con Git en dos commits.

Pista: si dos productos empatan en una tienda, decide qué hacer y déjalo documentado en un comentario.

---

10. Autoevaluación

1. ¿Qué diferencia hay entre `[1, 2, 3] * 2` y `np.array([1, 2, 3]) * 2`? — La primera repite la lista (`[1, 2, 3, 1, 2, 3]`); la segunda multiplica cada elemento (`[2, 4, 6]`).
2. ¿Qué es la vectorización y por qué importa? — Aplicar una operación al arreglo completo en lugar de elemento por elemento con un `for`; es más corta y mucho más rápida porque el bucle ocurre en código compilado.
3. Para `m = np.array([[1, 2, 3], [4, 5, 6]])`, ¿qué devuelve `m.sum(axis=0)`? — `[5, 7, 9]` (suma de cada columna).
4. ¿Qué hace `sueldos[sueldos > 3]`? — Devuelve solo los elementos mayores que 3, usando una máscara booleana.

5. ¿Qué es un `DataFrame` en una frase? — Una tabla con columnas con nombre, donde cada columna es un arreglo de un solo tipo.
6. ¿Por qué se rellenan los faltantes con la mediana y no con la media? — Porque la mediana no se deja arrastrar por valores atípicos (Semana 8).
7. Explica `groupby` sin jerga. — Separa las filas en grupos según una columna, hace un cálculo (suma, media…) por grupo y junta los resultados en una tabla.
8. V/F: "`np.std` y `Series.std` de Pandas dan siempre el mismo valor." — Falso: NumPy usa `ddof=0` (poblacional) por defecto y Pandas `ddof=1` (muestral).
9. ¿Qué tres elementos no pueden faltar en un gráfico? — Título, etiquetas de ejes (con unidades) y, si hay varias series, leyenda.

---

11. Verificación de dependencias

Depende de la Semana 8 (media, mediana, desviación estándar y normalización, que aquí se hacen con una línea; la semilla reproducible y la API `default_rng`), de la Semana 7 (vectores, producto punto con `@`, matrices como filas × columnas y `pytest.approx`), de la Semana 6 (pruebas con `pytest`, copias frente a mutación, entornos y versionado con Git en el ejercicio), de la Semana 5 (intuición de por qué ciertas operaciones escalan mejor) y de las Semanas 4 y 3 (listas, diccionarios, *list comprehensions*, funciones y anotaciones de tipo). Las librerías nuevas (NumPy, Pandas, Matplotlib) se introducen desde cero en esta clase, a partir de conceptos ya vistos; ningún tema posterior (scikit-learn, redes neuronales) es necesario. Es prerrequisito directo del Módulo 10 (Machine Learning clásico con scikit-learn, que consume arreglos y DataFrames) y del Módulo 11 (Deep Learning, cuyos tensores extienden los arreglos de NumPy).